



CELEBRATING 50 YEARS

PES UNIVERSITY

Department of Computer Applications



LAB MANUAL

System and Data Management

Linux Command-Line Navigation and File Operations

UQ26CA142A

BCA • Semester I

Academic Year 2026-27

Prepared By: Dilip Kumar Maripuri



Lab Overview

Objective

To navigate the Linux file-system hierarchy, understand the syntax and key options of the Session 06 Linux commands, create and manage files and directories, and perform basic text editing using nano and vi.

Learning Outcomes

- Identify the current working directory using pwd.
- List directory contents using ls and the discussed options.
- Navigate using absolute paths, relative paths, ~, .. and - with cd.
- Create directories and files using mkdir and touch.
- Copy, move, rename and remove files/directories using cp, mv and rm.
- Create and edit text files using nano and basic vi commands.

Before You Begin

Requirement	What to Check
Linux Environment	A working Ubuntu/Linux terminal is available.
Starting Account	Use your normal non-root Linux user account.
Lab Workspace	You will create ~/sdm_lab/session06 during this lab.
Safety	Run deletion commands only on files/directories created for this lab.

Command Syntax - Read This First

Linux command pattern

command [options] [arguments]

Example: ls -la /home/student

Option Pattern	Meaning	Examples
Single dash + letter	Short option	-l, -a, -h, -r, -i, -p
Combined single dash	Multiple short options together	-la, -lah
Double dash + word	Long-form option	--help, --version, --recursive

Case sensitivity

Linux commands and filenames are case-sensitive. 'ls' and 'LS' are different commands; 'Data.txt' and 'data.txt' are different files.



pwd - Print Working Directory

pwd displays the complete absolute path of the directory in which the shell is currently working.

Syntax

```
$ pwd
```

Command / Form	Meaning	When to Use	Example / Result
pwd	Print the current directory	Confirm where you are before file operations	\$ pwd -> /home/student

Important

Use pwd frequently while learning navigation. The output begins at / because it is an absolute path.

ls - List Directory Contents

ls lists files and directories. The Session 06 options increase the amount of detail, reveal hidden files, display readable sizes, or recurse through subdirectories.

Syntax

```
$ ls [options] [path]
```

Command / Form	Meaning	When to Use	Example / Result
ls	Names only	Quick look at current directory	\$ ls
ls -l	Long format: permissions, owner, size, date, name	Inspect file details	\$ ls -l
ls -la	Long format including names beginning with .	Show hidden configuration files	\$ ls -la
ls -lh	Long format with KB/MB/GB sizes	Read file sizes easily	\$ ls -lh
ls -lah	Combine -l, -a and -h	Detailed home-directory inspection	\$ ls -lah
ls -lR	Long format recursively through subdirectories	Inspect an entire directory tree	\$ ls -lR ~/sdm_lab
ls /etc	List a named path instead of current directory	Inspect another directory without cd	\$ ls /etc
ls -la /	Detailed listing of root directory	Explore top-level Linux directories	\$ ls -la /

Important

Use recursive listing with care on very large directory trees; the Session 06 source specifically warns against using it casually on /.



cd - Change Directory

cd changes the shell's current working directory. Session 06 uses absolute paths, relative paths, the home shortcut, parent references and previous-directory switching.

Syntax

```
$ cd [directory]
```

Command / Form	Meaning	When to Use	Example / Result
cd /	Go to root	Start at the top of the filesystem	\$ cd /
cd /var/log	Absolute path	Jump directly to a known location	\$ cd /var/log
cd bin	Relative path	Enter a child of the current directory	From /usr: cd bin -> /usr/bin
cd ..	Go to parent directory	Move one level upward	/usr/bin -> /usr
cd ../../	Go up two levels	Move upward multiple levels	Use relative parent references
cd ~	Go to home directory	Return to your own home	\$ cd ~
cd	Go to home directory	Same home behaviour without an argument	\$ cd
cd -	Return to previous directory	Toggle between recent locations	\$ cd -
cd ../etc	Relative path using parent then child	Navigate without starting at /	From /usr: ../etc -> /etc

Important

After a navigation command, run pwd whenever you need to verify the resulting location.



mkdir - Make Directory

mkdir creates directories. Session 06 uses normal directory creation and -p for nested paths.

Syntax

```
$ mkdir directory_name
```

```
$ mkdir -p parent/child/grandchild
```

Command / Form	Meaning	When to Use	Example / Result
mkdir sdm_lab	Create one directory	Create the main lab folder	Creates ~/sdm_lab when run from home
mkdir sdm_lab/session06	Create a subdirectory	Build the session folder	Requires parent to exist
mkdir -p sdm_lab/session06/practice/nested	Create all missing parent levels	Build nested paths in one command	Creates practice and nested as needed

Important

With -p, mkdir does not report an error merely because the directory already exists. This makes it useful for repeatable lab setup and scripts.

touch - Create Empty Files / Update Timestamp

touch is used in Session 06 to create empty files. When the named file already exists, touch updates its timestamp rather than creating a second file.

Syntax

```
$ touch file1.txt
$ touch file1.txt file2.txt notes.txt
```

Command / Form	Meaning	When to Use	Example / Result
touch file1.txt	Create one empty file	Prepare a file for later editing	New file initially has size 0
touch file1.txt file2.txt notes.txt	Create multiple files	Create several placeholders at once	Three files created in one command
touch existing.txt	Update an existing file timestamp	Observe modification-time behaviour	No duplicate file is created

Important

Verify file existence, size and timestamp with ls -la as done in Session 06.

cp - Copy Files and Directories

cp duplicates data. The source remains at its original location while a copy is created at the destination.

Syntax

```
$ cp [options] source destination
```

Command / Form	Meaning	When to Use	Example / Result
cp file.txt copy.txt	Copy a file in the same directory	Create a backup or duplicate	Both source and copy remain
cp file.txt /tmp/	Copy a file into a directory	Place a copy elsewhere	Source remains in current directory
cp -r directory/ destination/	Copy directory and all contents recursively	Duplicate a directory tree	Required for directories
cp -p file.txt copy.txt	Preserve permissions, ownership and timestamps	Retain source metadata	Copy keeps metadata
cp -i file.txt dest.txt	Prompt before overwriting	Safer when destination may exist	User confirms overwrite

Important

Session 06 verifies copies with ls -la and checks that a recursively copied directory contains the same subdirectory structure.



mv - Move and Rename

mv has two roles: rename an item when source and destination are in the same directory, or move it when the destination is another directory.

Syntax

```
$ mv [option] source destination
```

Command / Form	Meaning	When to Use	Example / Result
mv file2.txt renamed_file2.txt	Rename a file	Change a filename	Old name disappears; new name appears
mv renamed_file2.txt practice/	Move a file	Relocate into another directory	File appears inside practice/
mv practice_backup/ ~/sdm_lab/	Move a directory	Relocate a complete directory	Directory moves to parent location
mv -i source dest	Prompt before overwriting	Safer move/rename when destination may exist	User confirmation before overwrite

Important

The Session 06 source warns that mv can overwrite an existing destination and there is no normal undo. Use mv -i when overwrite risk exists.

rm - Remove Files and Directories

rm permanently removes files. Recursive forms can remove complete directory trees and must be used carefully.

Syntax

```
$ rm [options] file_or_directory
```

Command / Form	Meaning	When to Use	Example / Result
rm file.txt	Delete one file	Remove a known file	Permanent deletion
rm -i file.txt	Prompt before deleting	Safest form for beginners	Type y only after checking the filename
rm file1.txt file2.txt	Delete multiple files	Remove several named files	All listed files are targeted
rm -r directory/	Delete directory and all contents recursively	Remove a test directory tree	High caution; irreversible
rm -rf directory/	Force recursive removal without prompts	Only when explicitly intended	Very high risk; no confirmation

Important

Never run rm -rf / . The Session 06 material labels it system-destroying. Linux rm has no recycle bin, confirmation, or undo when force is used.

cat - Verify Saved Text

In this Session 06 manual, cat is used only to display the contents of files created with nano or vi so students can verify that saving succeeded.

Syntax

```
$ cat filename
```

Command / Form	Meaning	When to Use	Example / Result
cat my_notes.txt	Display file contents in terminal	Verify nano saved correctly	\$ cat ~/sdm_lab/session06/my_notes.txt
cat vi_practice.txt	Display file contents in terminal	Verify vi saved correctly	\$ cat ~/sdm_lab/session06/vi_practice.txt

Important

No additional cat options are introduced here because the lab is restricted to the Session 06 command domain.

nano - Elaborated Basic Commands

nano is the beginner-friendly terminal editor used in Session 06. Its shortcut bar uses ^ to mean the Ctrl key. For example, ^O means Ctrl+O.

```
$ nano ~/sdm_lab/session06/my_notes.txt
```

Key / Action	Function	What You Observe	Typical Use
Type normally	Insert text at the cursor	Characters appear immediately	Write file content
Arrow keys	Move the cursor	Cursor moves without changing text	Navigate to edit location
Ctrl+O, then Enter	Write Out (save) without exiting	nano asks for filename; Enter confirms	Save progress
Ctrl+X	Exit nano	If unsaved, nano prompts about saving	Finish editing
Ctrl+K	Cut current line	Current line is removed and held for paste	Delete or move a line
Ctrl+U	Paste the cut line	Previously cut line reappears	Paste after Ctrl+K
Ctrl+W	Search (Where Is)	nano asks for search text	Find text in file
Ctrl+G	Help	Help screen opens	View nano commands

nano save-and-exit sequence

Type text -> Ctrl+O -> Enter -> Ctrl+X. Then verify with cat ~/sdm_lab/session06/my_notes.txt.



vi - Elaborated Basic Commands

vi is presented here as a modal editor. Students must know which mode they are in before typing commands. The basic modal commands below are the same set practised in the Session 06 source under vim.

Mode	How to Enter	What Keystrokes Do	Indicator / Behaviour
Normal	Esc	Navigate, delete, undo, issue editing commands	No INSERT indicator
Insert	i or a	Type text normally	INSERT indicator appears
Command	From Normal, press :	Save and quit commands such as :w, :q, :wq	Colon command appears at bottom
Visual	From Normal, press v	Select text	VISUAL indicator appears

```
$ vi ~/sdm_lab/session06/vi_practice.txt
```

Goal	Command	Mode	Notes
Enter Insert mode	i	Normal -> Insert	Insert before cursor
Append text	a	Normal -> Insert	Append after cursor
Return to Normal	Esc	Insert/Visual -> Normal	Use Esc whenever unsure of mode
Save file	:w + Enter	Normal -> Command	w = write
Quit	:q + Enter	Normal -> Command	Works when there are no unsaved changes
Save and quit	:wq + Enter	Normal -> Command	Most common exit sequence
Quit without saving	:q! + Enter	Normal -> Command	! forces exit; unsaved changes are lost
Move cursor	h / j / k / l	Normal	Left / down / up / right; arrow keys also work
Delete current line	dd	Normal	Removes current line
Undo last action	u	Normal	Repeated u undoes multiple actions
Search	/searchterm + Enter	Normal	Search forward for text
Next / previous match	n / N	Normal	Move among search matches

vi survival sequence

Open file -> i -> type text -> Esc -> :wq -> Enter -> verify using cat. If you become unsure which mode you are in, press Esc first.



File-System Navigation

1 Identify the current directory with pwd

pwd prints the complete path of your current working directory. Run it whenever you are unsure where you are.

Ubuntu Terminal

```
$ pwd
```

Typical output

Expected Output

```
/home/student
```

Observed output / remarks

2 List directory contents with ls

Run ls with increasing detail. Compare how each option changes the output.

Command	What It Shows	When to Use It
ls	Filenames only	Quick glance at contents
ls -l	Long format: permissions, owner, size, date, name	See file details
ls -la	Long format including hidden files	See hidden/config files
ls -lh	Long format with human-readable sizes	Check file sizes
ls -lR	Long format recursively through subdirectories	See a directory tree
ls /etc	Contents of /etc	List another directory by path

Ubuntu Terminal

```
$ ls
$ ls -l
$ ls -la
$ ls -lah
$ ls -la /
```



Representative output

Expected Output

```
$ ls
Desktop Documents Downloads Music Pictures Videos

$ ls -la
total 48
drwxr-x--- 12 student student 4096 Jul 4 09:15 .
drwxr-xr-x  3 root   root    4096 Jul 4 08:00 ..
-rw-r--r--  1 student student  220 Jul 4 08:00 .bash_logout
-rw-r--r--  1 student student 3526 Jul 4 08:00 .bashrc
```

Reading ls -la

The first character identifies the entry type ('d' directory, '-' regular file). The special entries '.' and '..' represent the current and parent directory.

Column in ls -l Output	What It Means	Example from Your Output
First character (d or -)	Entry type	
Characters 2-10	Permission bits	
Number after permissions	Link count	
First name	Owner	
Second name	Group	
Number such as 4096	Size	
Date and time	Last modification	
Last field	Filename	

3 Change directories with cd

cd accepts absolute paths, relative paths, and special shortcuts.

Command	Where It Goes	Type
cd /	Root directory /	Absolute path
cd /home/student	Student home directory	Absolute path
cd Documents	Documents inside current directory	Relative path



Command	Where It Goes	Type
cd ..	Parent directory	Relative path
cd ../../	Two levels up	Relative path
cd ~ or cd	Home directory	Shortcut
cd -	Previous working directory	Shortcut

Ubuntu Terminal

```
$ cd /  
$ pwd  
$ cd /usr  
$ cd bin  
$ pwd  
$ cd ..  
$ cd ../etc  
$ pwd  
$ cd ~  
$ pwd  
$ cd -
```

Observed output / remarks

Path reminder

Absolute paths begin at '/'. Relative paths begin from your current directory. '..' means parent; '~' means home; 'cd -' toggles back to the previous directory.

Directory and File Operations

4 Create the lab workspace with mkdir

Ubuntu Terminal

```
$ cd ~  
$ mkdir sdm_lab  
$ mkdir sdm_lab/session06  
$ mkdir -p sdm_lab/session06/practice/nested  
$ ls -R sdm_lab
```



Expected structure

Expected Output

```
sdm_lab:
session06

sdm_lab/session06:
practice

sdm_lab/session06/practice:
nested
```

mkdir -p

The -p option creates missing parent directories and does not report an error if the target directory already exists.

5 Create files with touch

Ubuntu Terminal

```
$ cd ~/sdm_lab/session06
$ touch file1.txt file2.txt notes.txt
$ ls -la
$ touch existing.txt
$ ls -la existing.txt
```

Observe that newly created files initially have size 0. Running touch on an existing file updates its modification timestamp.

Observed output / remarks

6 Copy files and directories with cp

Command	What It Does
cp file.txt copy.txt	Copy within the same directory
cp file.txt /tmp/	Copy into /tmp
cp -r directory/ destination/	Copy a directory recursively
cp -p file.txt copy.txt	Preserve permissions, ownership and timestamps
cp -i file.txt dest.txt	Prompt before overwriting



Ubuntu Terminal

```
$ cd ~/sdm_lab/session06
$ cp file1.txt file1_backup.txt
$ cp file1.txt /tmp/
$ cp -r practice/ practice_backup/
$ ls -la
$ ls -la practice_backup/
```

Checkpoint

practice_backup/ should exist and contain the same subdirectory structure as practice/.

7 Move and rename with mv

Ubuntu Terminal

```
$ cd ~/sdm_lab/session06
$ mv file2.txt renamed_file2.txt
$ ls
$ mv renamed_file2.txt practice/
$ ls practice/
$ mv practice_backup/ ~/sdm_lab/
```

Caution

mv can overwrite an existing destination. The discussed safer form is 'mv -i', which prompts before overwriting. There is no built-in undo for mv.

8 Remove files and directories with rm

Command	Effect	Caution
rm file.txt	Delete one file	Moderate
rm -i file.txt	Prompt before deleting	Safest
rm file1.txt file2.txt	Delete multiple files	Moderate
rm -r directory/	Delete directory and all contents recursively	High
rm -rf directory/	Force recursive delete without prompts	Very high



Ubuntu Terminal

```
$ cd ~/sdm_lab/session06
$ rm -i file1_backup.txt
$ mkdir temp_delete && touch temp_delete/a.txt temp_delete/b.txt
$ rm -r temp_delete/
$ ls
```

Safety warning

rm -rf has no recycle bin, confirmation, or undo. Never run 'rm -rf /'. Always double-check the path before pressing Enter.

Text Editors - nano and vi

9 Create and edit a file with nano

Ubuntu Terminal

```
$ nano ~/sdm_lab/session06/my_notes.txt
```

nano is a simple terminal text editor. Use the basic commands below to type, move, save, search, cut/paste, and exit. Then enter the following text exactly:

Text to Enter

```
SDM Lab Session 06 - My First Linux File
Student Name: _____
Date: _____
```

Key commands learned today:

1. pwd - print working directory
2. ls - list directory contents
3. cd - change directory
4. mkdir / rm / cp / mv



Press Ctrl+O to save, then Enter to confirm the filename. Press Ctrl+X to exit nano.

Basic nano Command	What It Does	When to Use It
Type normally	Insert text at cursor	Writing content
Arrow keys	Move cursor	Navigate in file
Ctrl+O then Enter	Save (Write Out)	Save without exiting
Ctrl+X	Exit nano	When finished
Ctrl+K	Cut current line	Delete/move a line
Ctrl+U	Paste cut line	After Ctrl+K
Ctrl+W	Search	Find text
Ctrl+G	Help	View nano help

Verification

```
$ cat ~/sdm_lab/session06/my_notes.txt
```

Observed output / remarks

10 Complete the basic vi survival sequence

vi is a modal text editor. It starts in NORMAL mode. Press i or a to enter INSERT mode, press Esc to return to NORMAL mode, and use colon commands such as :w, :q, and :wq to save or quit.

Ubuntu Terminal

```
$ vi ~/sdm_lab/session06/vi_practice.txt
```

Step	Action	Key / Command
1	Enter Insert mode	i
2	Type: vi is a modal editor. I am in INSERT mode.	Type normally
3	Return to Normal mode	Esc
4	Save and quit	:wq then Enter
5	Verify saved file	cat vi_practice.txt



Basic vi Command	Purpose
i	Enter Insert mode
a	Append after cursor
Esc	Return to Normal mode
:w	Save
:q	Quit
:wq	Save and quit
:q!	Quit without saving
h j k l	Move cursor left/down/up/right
dd	Delete current line
u	Undo
/searchterm	Search
n / N	Next / previous match

Verification

```
$ cat ~/sdm_lab/session06/vi_practice.txt
```

Observed output / remarks



Independent Lab Exercises

Lab assessment note

Complete these exercises independently. Record every command you run and the output you receive.

Exercise 1 - Directory Tree Construction

Construct the following exact directory structure under `~/sdm_lab/` using a single `mkdir` command with `-p`:

Target Structure

```
~/sdm_lab/  
unit1/  
  theory/  
  labs/  
unit2/  
  exercises/
```

Command Used	Output of <code>ls -R ~/sdm_lab/</code>

Exercise 2 - File Operations Chain

Task	Command Used	Observed Result
Create <code>kernel_notes.txt</code> inside <code>~/sdm_lab/unit1/theory/</code>		
Open it in nano and enter: Linux uses a Monolithic kernel with LKMs.		
Copy it to <code>~/sdm_lab/unit1/labs/</code> as <code>kernel_notes_copy.txt</code>		
Move the copy into <code>~/sdm_lab/unit2/</code> and rename it <code>unit2_reference.txt</code>		
Delete the original <code>kernel_notes.txt</code>		
Verify final directory contents with <code>ls</code>		



Exercise 3 - Navigation Challenge

Target	Constraint	Command(s) Used	pwd Output
/usr/bin	Navigate using cd		
/etc/apt	Use absolute and relative navigation		
Home directory	Use home shortcut		
/var/log	Use an absolute path		
Previous directory	Use cd -		

Exercise 4 - ls Exploration

Question	Command Used	Your Answer / Output
How many files and directories are in /etc/?		
Which file in /home/student/ is the largest?		
List all hidden files in your home directory.		
What is the size (in bytes) of /bin/bash?		
How many items are in /usr/bin/?		

Post-Lab Questions and Reflection

#	Question	Your Answer
1	When you ran 'ls -la /dev', entries can begin with 'b' or 'c'. Which OS concept from Sessions 01/03 does /dev demonstrate?	
2	The rm command has no undo. Which OS function is responsible for file deletion and reclaiming disk space?	
3	vi has Normal, Insert and Command modes. Which OS concept from Session 03 is this compared with in the source material?	
4	When you run 'mkdir new_dir', which OS service from Session 04 is being invoked?	



Final Verification

Complete the following checks and record the observed result.

#	Command	Expected Check	Observed / Remarks
1	pwd	Current path displayed	
2	ls -lah	Hidden files and human-readable details shown	
3	ls -R ~/sdm_lab/	Lab directory tree displayed	
4	cat ~/sdm_lab/session06/my_notes.txt	Nano file content displayed	
5	cat ~/sdm_lab/session06/vi_practice.txt	vi file content displayed	

Quick Reference

Purpose	Command
Show current directory	pwd
List contents	ls
Long listing	ls -l
Show hidden files	ls -la
Human-readable long listing	ls -lah
Recursive long listing	ls -lR
Go to root	cd /
Go home	cd ~ or cd
Go to parent	cd ..
Return to previous directory	cd -
Create directory	mkdir dirname
Create nested directories	mkdir -p path/to/dir
Create empty file	touch filename
Copy file	cp source dest
Copy directory recursively	cp -r source/ dest/
Preserve metadata while copying	cp -p source dest
Prompt before copy overwrite	cp -i source dest
Rename/move	mv source dest
Prompt before mv overwrite	mv -i source dest
Remove file interactively	rm -i filename
Remove directory recursively	rm -r directory/
Open nano	nano filename
Verify text file	cat filename



SYSTEM AND DATA MANAGEMENT

UQ26CA142A | BCA • Semester I | Dept. of Computer Applications
Linux Command-Line Navigation and File Operations - Lab Manual



CELEBRATING 50 YEARS

Purpose	Command
Open vi	vi filename
vi insert mode	i
vi save and quit	Esc, then :wq

Lab note

Commands in this manual are intentionally restricted to the Session 06 command-line navigation, file/directory operation, nano, vi, and cat-verification commands. Installation procedures and later-session permission/process utilities are excluded.